

Студент (име и презиме):
Број индекса:
14. септембар 2026.

Факултет инжењерских наука
Практикум из рачунарских алата
Испит

Задаци:

У свим задацима важи претпоставка да се наредбе/програми/скрипте извршавају у **bash** окружењу. Наредбе које извршавају корисници обавезно започињати са **\$**, а које извршава суперкорисник са **#**.

Задатак 1 – [20%]

[2%] — Приказати вредност променљиве окружења **HOME**.

1 решење

[2%] — Приказати последњих 20 линија датотеке **server.log**.

1 решење

[2%] — Сортирати линије датотеке **imena.txt** и уклонити дубликате.

1 решење

[2%] — Из датотеке **korisnici.csv** издвојити другу колону, при чему је зарез сепаратор.

1 решење

[2%] — Направити датотеке **z1.txt**, **z2.txt**, ..., **z5.txt** једном наредбом.

1 решење

[2%] — Пронаћи празне регуларне датотеке у тренутном радном директоријуму и његовим поддиректоријумима.

1 решење

[2%] — У датотеци **dco.v** заменити сва појављивања речи **tiniv** речју **cts** тако да датотека након наредбе буде измењена (опција *in place*).

1 решење

[2%] — Исписати само јединствене линије из већ сортиране датотеке **imena.txt**.

1 решење

[2%] — Променити власничку групу датотеке **projekat.zip** у **studenti**.

1 решење

[2%] — Стандардне грешке програма **provera.sh** уписати у датотеку **greske.log**.

1 решење

Задатак 2 – [40%]

[20%] — Написати **bash** скрипту која прима један аргумент – путању до текстуалне датотеке. Скрипта треба да:

- провери број аргумената,
- провери да датотека постоји и да није празна,
- испише на стандардном излазу број линија, број речи и број карактера,
- резултат упише и у датотеку **statistika.txt** у следећем формату:

Линије: <број линија>

Речи: <број речи>

Карактери: <број карактера>

решење

```

1  #!/usr/bin/env bash
2
3  if [[ $# -ne 1 ]]; then
4      echo "Грешка: Потребан је један аргумент."
5      exit 1
6  fi
7
8  f="$1"
9
10 if [[ ! -f "$f" ]]; then
11     echo "Грешка: Датотека не постоји."
12     exit 2
13 fi
14
15 if [[ ! -s "$f" ]]; then
16     echo "Грешка: Датотека је празна."
17     exit 3
18 fi
19
20 linije=$(wc -l < "$f")
21 reci=$(wc -w < "$f")
22 karakteri=$(wc -m < "$f")
23
24 echo "Линије: $linije"
25 echo "Речи: $reci"
26 echo "Карактери: $karakteri"
27
28 {
29     echo "Линије: $linije"
30     echo "Речи: $reci"
31     echo "Карактери: $karakteri"
32 } > statistika.txt
33
34
35
36
37
38
39
40
41
42
43
44
45
46

```

решење

[20%] — Написати `bash` скрипту која као једини аргумент прима непразну текстуалну датотеку чији редови могу садржати ознаке `INFO`, `WARNING` или `ERROR`. Скрипта треба да:

- провери број аргумената, постојање датотеке и да ли је она непразна;
- све линије које садрже `ERROR` упише у датотеку `errors.txt`;
- испише на стандардном излазу број линија са сваком од три ознаке као и укупан број линија полазне датотеке у следећем формату:

```
INFO: <број линија које садрже ознаку INFO>
WARNING: <број линија које садрже ознаку WARNING>
ERROR: <број линија које садрже ознаку ERROR>
Укупно: <укупан број линија>
```

решење

```
1  #!/usr/bin/env bash
2
3  if [[ $# -ne 1 ]]; then
4      echo "Употреба: $0 <log_датотека>"
5      exit 1
6  fi
7
8  datoteka=$1
9
10 if [[ ! -f "$datoteka" ]]; then
11     echo "Грешка: Датотека не постоји."
12     exit 2
13 fi
14
15 if [[ ! -s "$datoteka" ]]; then
16     echo "Грешка: Датотека је празна."
17     exit 3
18 fi
19
20 grep 'ERROR' "$datoteka" > greske.txt
21
22 echo "INFO: $(grep -c 'INFO' "$datoteka")"
23 echo "WARNING: $(grep -c 'WARNING' "$datoteka")"
24 echo "ERROR: $(grep -c 'ERROR' "$datoteka")"
25 echo "Укупно: $(wc -l < "$datoteka")"
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
```

решење

Задатак 3 – [20%]

[20%] — У новом директоријуму /srv/redakcija направити поддиректоријуме tekstovi и objavljeno. Направити групе autori и urednici. Кориснике ana и marko додати у групу autori, а кориснике jelena и petar у групу urednici. Свим корисницима подесити лозинке.

Власник оба директоријума треба да буде root. Власничка група директоријума tekstovi треба да буде autori, а директоријума objavljeno група urednici. Власник и група имају сва права. Остали немају права над tekstovi, док над objavljeno имају право читања и приступа. Нове датотеке треба да наслеђују групу директоријума.

решење

```
1      # mkdir -p /srv/redakcija/{tekstovi,objavljeno}
2
3      # groupadd autori
4      # groupadd urednici
5
6      # useradd ana
7      # usermod -aG autori ana
8      # passwd ana
9
10     # useradd marko
11     # usermod -aG autori marko
12     # passwd marko
13
14     # useradd jelena
15     # usermod -aG urednici jelena
16     # passwd jelena
17
18     # useradd petar
19     # usermod -aG urednici petar
20     # passwd petar
21
22     # chown root:autori /srv/redakcija/tekstovi
23     # chown root:urednici /srv/redakcija/objavljeno
24
25     # chmod 2770 /srv/redakcija/tekstovi
26     # chmod 2775 /srv/redakcija/objavljeno
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

решење

Задатак 4 – [20%]

[2%] — Приказати адресе свих удаљених репозиторијума.

1 `$ git remote -v` решење _____
_____ решење _____

[2%] — Преузети информације о изменама са репозиторијума `origin` без спајања.

1 `$ git fetch origin` решење _____
_____ решење _____

[2%] — Направити грану `izvestaj` и одмах се пребацити на њу.

1 `$ git switch -c izvestaj` решење _____
2 или _____
3 `$ git checkout -b izvestaj` _____
_____ решење _____

[2%] — Приказати статус репозиторијума у скраћеном формату.

1 `$ git status -s` решење _____
_____ решење _____

[2%] — Додати све `.tex` датотеке у индекс.

1 `$ git add *.tex` решење _____
_____ решење _____

[2%] — Направити комит са поруком Додати извештаји.

1 `$ git commit -m "Додати извештаји"` решење _____
_____ решење _____

[2%] — Приказати граф историје свих грана у скраћеном облику.

1 `$ git log --graph --oneline --all` решење _____
_____ решење _____

[2%] — Занемарити последњи комит и врати стање репозиторијума на претпоследњи комит.

1 `$ git reset --hard HEAD~1` решење _____
_____ решење _____

[2%] — Покренути алат за разрешавање конфликта приликом спајања који се обично покреће након што наредба `git merge` не успе аутоматски да споји измене у гранама..

1 `$ git mergetool` решење _____
_____ решење _____

[2%] — Налазите се на грани `feature`. Потребно је да пребаците све своје комитове на врх најновијег стања гране `main` ради линеарне историје.

1 `$ git rebase main` решење _____
_____ решење _____